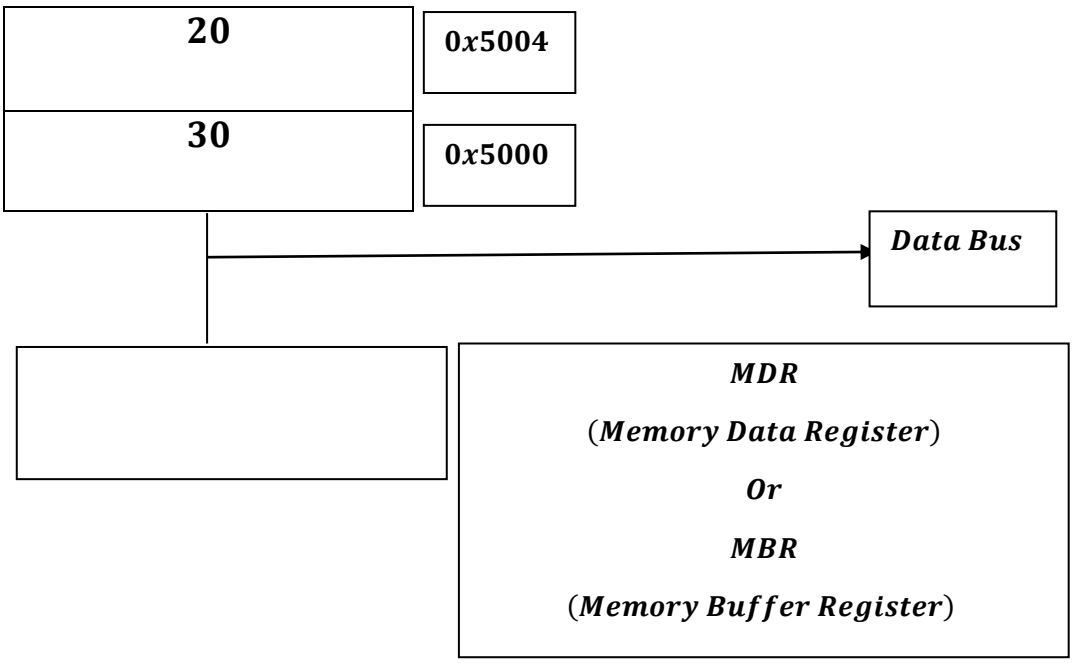
$Postfix\ Evaluation - Memory\ Mechanism$

$Stack - push\ and\ pop\ is\ represented\ logically\ through\ program.$

$In\ actual\ Memory\ mechanism$:

$In\ Stack$:



$\therefore\ We\ have\ to\ Add\ (20, 30)\ what\ will\ happen?$

$$20_{10}\ in\ binary = 10100_2\ \ and\ 30_{10}\ in\ binary = 11110_2$$

$\underline{In\ 16\ bit\ Memory}$

$With\ Opcode\ or\ Operational\ Code\ , assigned\ by\ CPU\ and\ it\ is\ of\ 4\ bit.$

$It\ is\ stored\ in\ Memory : 4 - bit\ Opcode + 12\ bit\ Operand.$

$For\ Push\ OpCode$: 0010

$For\ Add\ OpCode$: 0001

$12\ bit\ Representation\ of\ 20$: 000000010100

$12\ bit\ Representation\ of\ 30$: 000000011110

$4 - bit\ OpCode(Push) + 12 - bit\ Operand(20_{10})$: $0010\ 000000010100$

$4 - bit\ OpCode(Push) + 12 - bit\ Operand(30_{10})$: $0010\ 000000011110$

$4 - bit\ OpCode(Add) + 12 - bit\ Operand(0_{10})$: $0001\ 000000000000$

$i.e.\ in\ memory\ it\ will\ be\ stored\ in\ memory\ as\ binary\ format.$

$$In\ 32 - Bit\ it\ will\ become:$$

$$4 - bit\ Opcode + 12\ bit\ Operand + 16\ bit\ Unused(Zeroes)$$

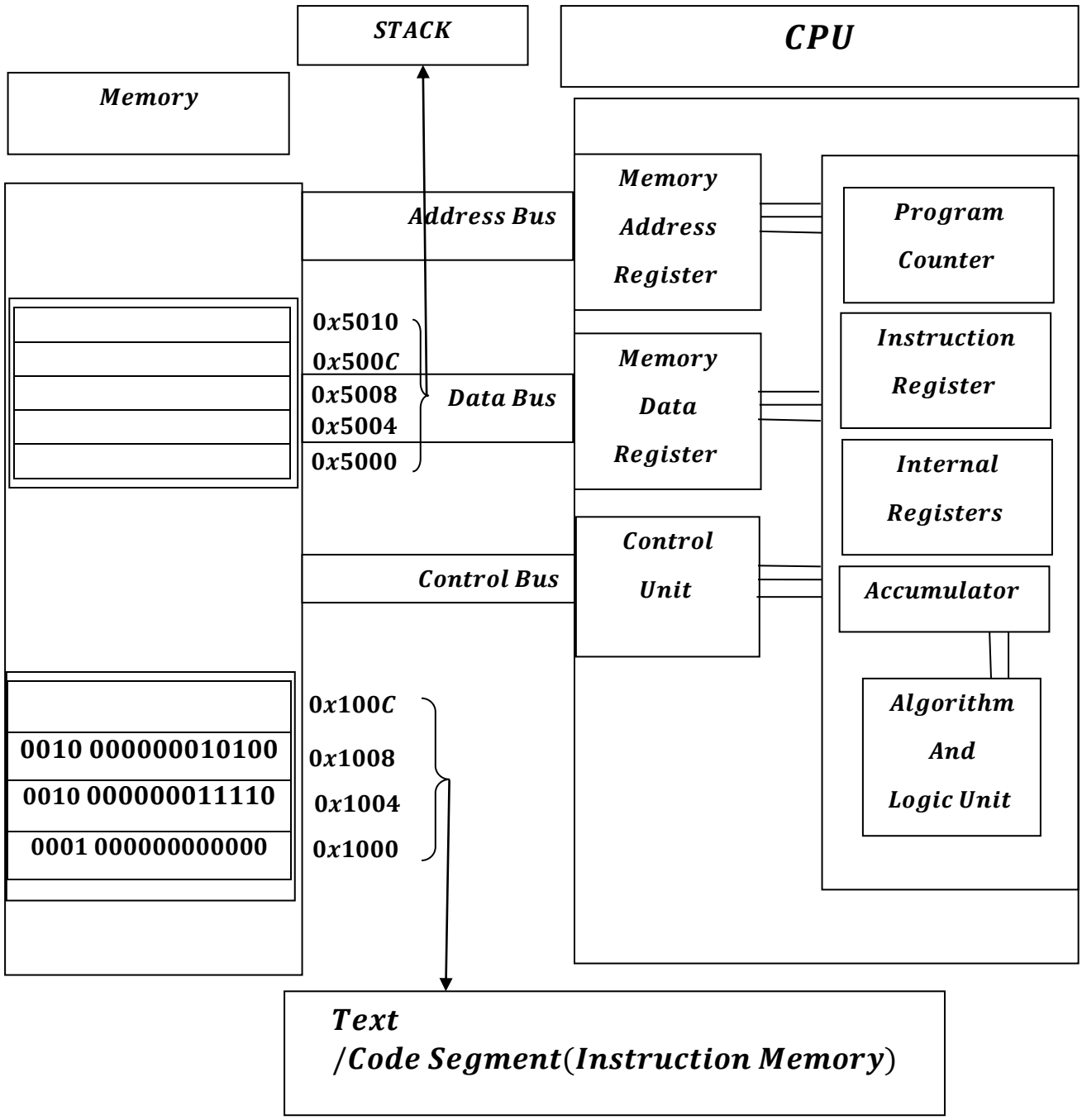
$$(20_{10}) = 0010\ 000000010100\ 0000000000000000$$

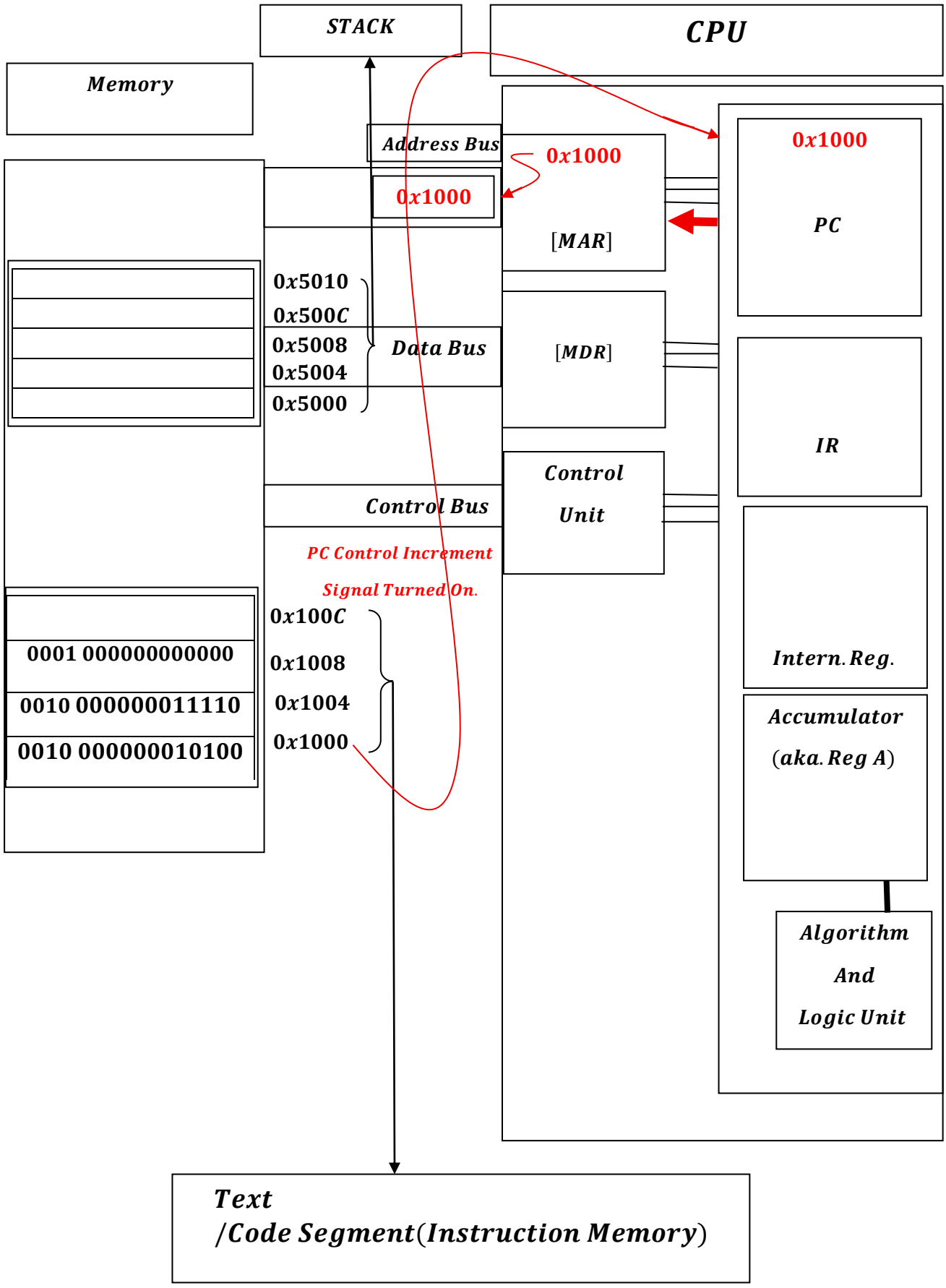
$$In\ 64 - Bit\ it\ will\ become:$$

$$4 - bit\ Opcode + 12\ bit\ Operand + 48\ bit\ Unused(Zeroes)$$

$Lets\ use\ 16-bit\ , though\ memory\ representation\ is$
$32-bit, [as\ an\ example\ only]\ and\ this$:

$4-bit\ Opcode+12\ bit\ Operand-will\ be\ stored\ not\ in\ stack\ but$
$inside: Text\ /Code\ Segment(Instruction\ Memory)\ \ of\ the\ memory$.




STACK
CPU
Memory
Address Bus
0x1000
0x1000
[MAR]
0x1000
PC
0x5010
0x500C
0x5008
0x5004
0x5000
Data Bus
[MDR]
IR
Control Unit
Control Bus
PC Control Increment
Signal Turned On.
0x100C
0x1008
0x1004
0x1000
0001 000000000000
0010 000000011110
0010 000000010100
Intern. Reg.
Accumulator
(aka. Reg A)
Algorithm
And
Logic Unit
Text
/Code Segment(Instruction Memory)

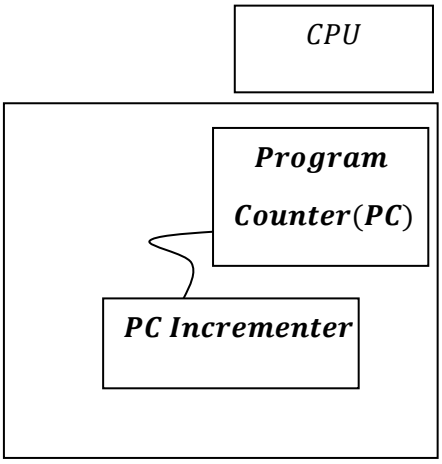
1. *When computer turns on Program Counter is loaded with predefined ADDRESS, and CPU is in RESET state, and this Address is RESET VECTOR ADDRESS* .

2. *As PC is with RESET VECTOR ADDRESS, CPU fetches the first instruction, Boot loader runs and the operating system loads*.

3. *If code segment starts at* $0x1000$, *then during reset PC holds* $0x1000$.

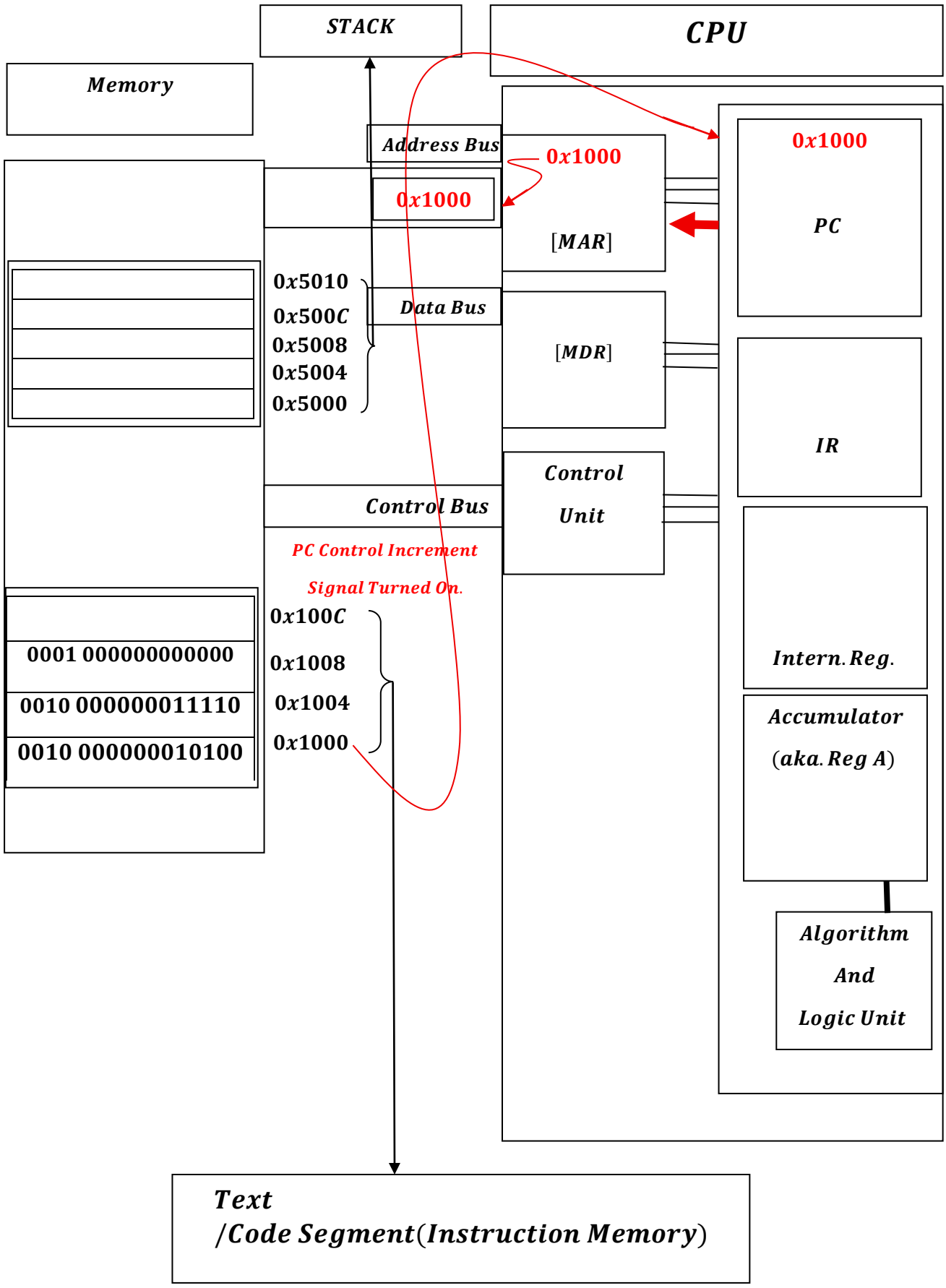
Then, when execution begins: $0x1000 \rightarrow 0x1004 \rightarrow 0x1008$.

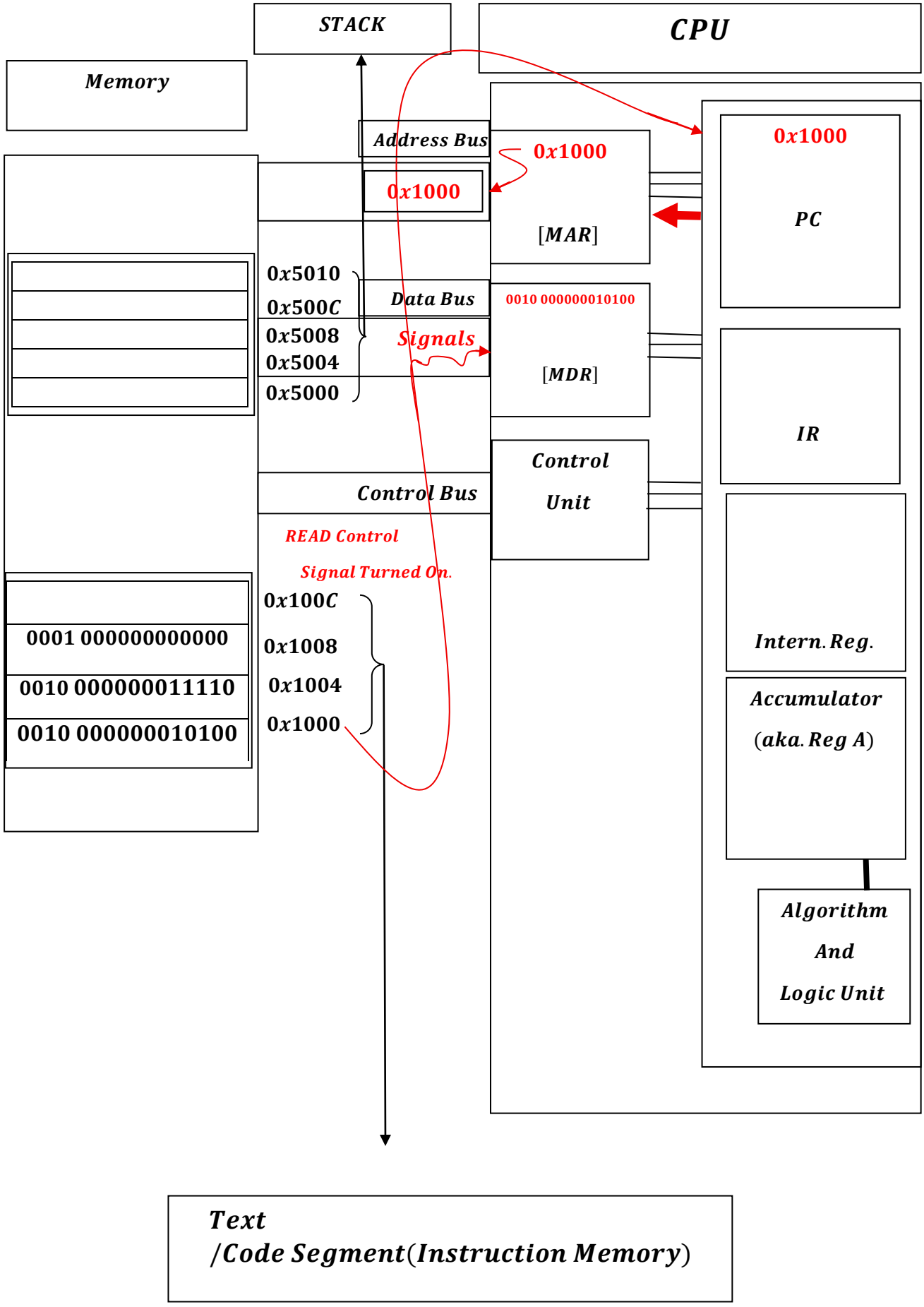
How does PC increments ?



This PC Incrementer or PC Adder adds a fixed size to the Address to have the next updated Address , say Address is $0x1000$, *it adds* $+4$, *to have the next address*: $0x1004$. *Note CU(Control Unit)triggers a PC Control Increment Signal for PC to increment*.

ALU(Arithmetic Logic Unit)uses: *Base Address* $+$ *index* $\times$ *sizeof(dataTypes) formula, when code asks to find a specific piece of data inside an array but not PC(Program Counter)*.


STACK
CPU
Memory
Address Bus
0x1000
0x1000
[MAR]
0x1000
PC
0x5010
0x500C
0x5008
0x5004
0x5000
Data Bus
[MDR]
IR
Control Unit
Control Bus
PC Control Increment
Signal Turned On.
0x100C
0x1008
0x1004
0x1000
0001 000000000000
0010 000000011110
0010 000000010100
Intern. Reg.
Accumulator
(aka. Reg A)
Algorithm
And
Logic Unit
Text
/Code Segment(Instruction Memory)


STACK
CPU
Memory
Address Bus
0x1000
0x1000
[MAR]
0x1000
PC
0x5010
0x500C
0x5008
0x5004
0x5000
Data Bus
Signals
0010 000000010100
[MDR]
IR
Control Unit
Control Bus
READ Control
Signal Turned On.
0x100C
0x1008
0x1004
0x1000
0001 000000000000
0010 000000011110
0010 000000010100
Intern. Reg.
Accumulator
(aka. Reg A)
Algorithm
And
Logic Unit
Text
/Code Segment(Instruction Memory)

$\rightarrow$ *The MAR places the address* $0x1000$ *onto the Address Bus. This bus is hardwired directly into the Memory Unit's input pins.*

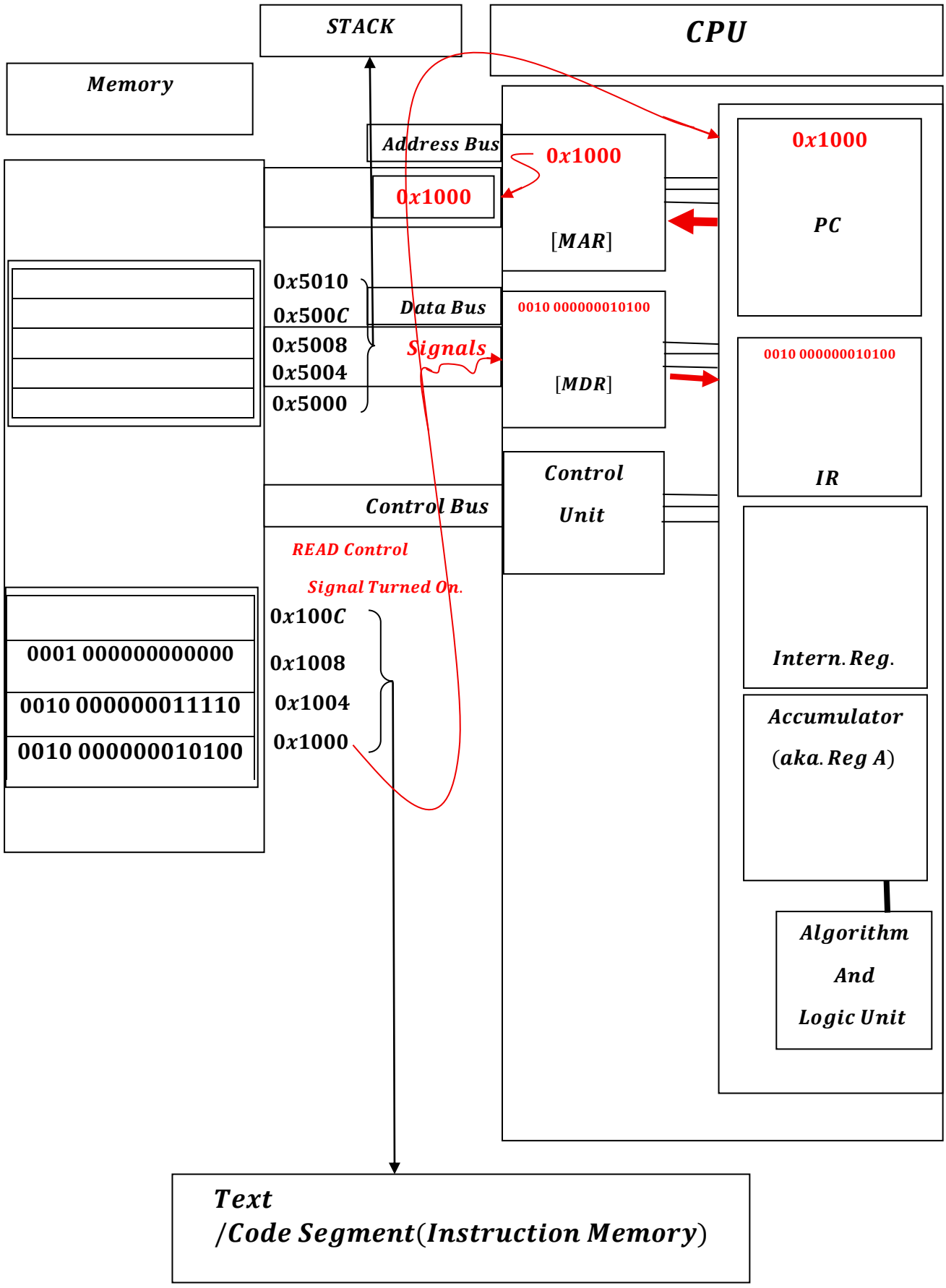
$\rightarrow$ *Control Bus*: *At the exact same time, the Control Unit sends a high − voltage electrical signal* (1) *down the READ wire of the Control Bus. This tells the Memory Unit ,"Wake up, look at the Address Bus, and prepare to send data out."*

$\rightarrow$ *The Lookup* (*Inside the Memory*): *This is where the magic happens. Inside the physical Memory Unit, there is a piece of hardware called an Address Decoder. It reads* $0x1000$ *from the Address Bus, navigates its vast grid of transistors, and physically connects to that specific row of memory cells.*

$\rightarrow$ *The Dispatch* (*Data Bus*): *Because the READ signal is active, the Memory Unit takes the binary data stored at* $0x1000 -$ 0010 000000010100 *− and pushes those electrical signals onto the Data Bus.*

$\rightarrow$ *The Catch* (*MDR*): *The Data Bus carries this instruction straight back to the CPU. The CPU waits a tiny fraction of a microsecond* (*to ensure the electrical signals on the bus have stabilized*), *and then the Control Unit triggers the MDR to "latch" or save whatever is currently on the Data Bus.*

The next , the Data stored in MDR moves via internal CPU wiring to INSTRUCTION REGISTER(*IR*).


STACK
CPU
Memory
Address Bus
0x1000
0x1000
[MAR]
0x1000
PC
0x5010
0x500C
0x5008
0x5004
0x5000
Data Bus
Signals
0010 000000010100
[MDR]
0010 000000010100
IR
Control Unit
Control Bus
READ Control
Signal Turned On.
0x100C
0x1008
0x1004
0x1000
0001 000000000000
0010 000000011110
0010 000000010100
Intern. Reg.
Accumulator
(aka. Reg A)
Algorithm
And
Logic Unit
Text
/Code Segment(Instruction Memory)

Inside the CPU

Memory Data Reg. (MDR)

(Just caught the data from the bus)

***Holds*: 0010 000000010100**

(Data moves via internal CPU wiring)

Instruction Register (IR)

(The official holding area for the current instruction)

***Holds*: 0010 000000010100**

(Data is read and split)

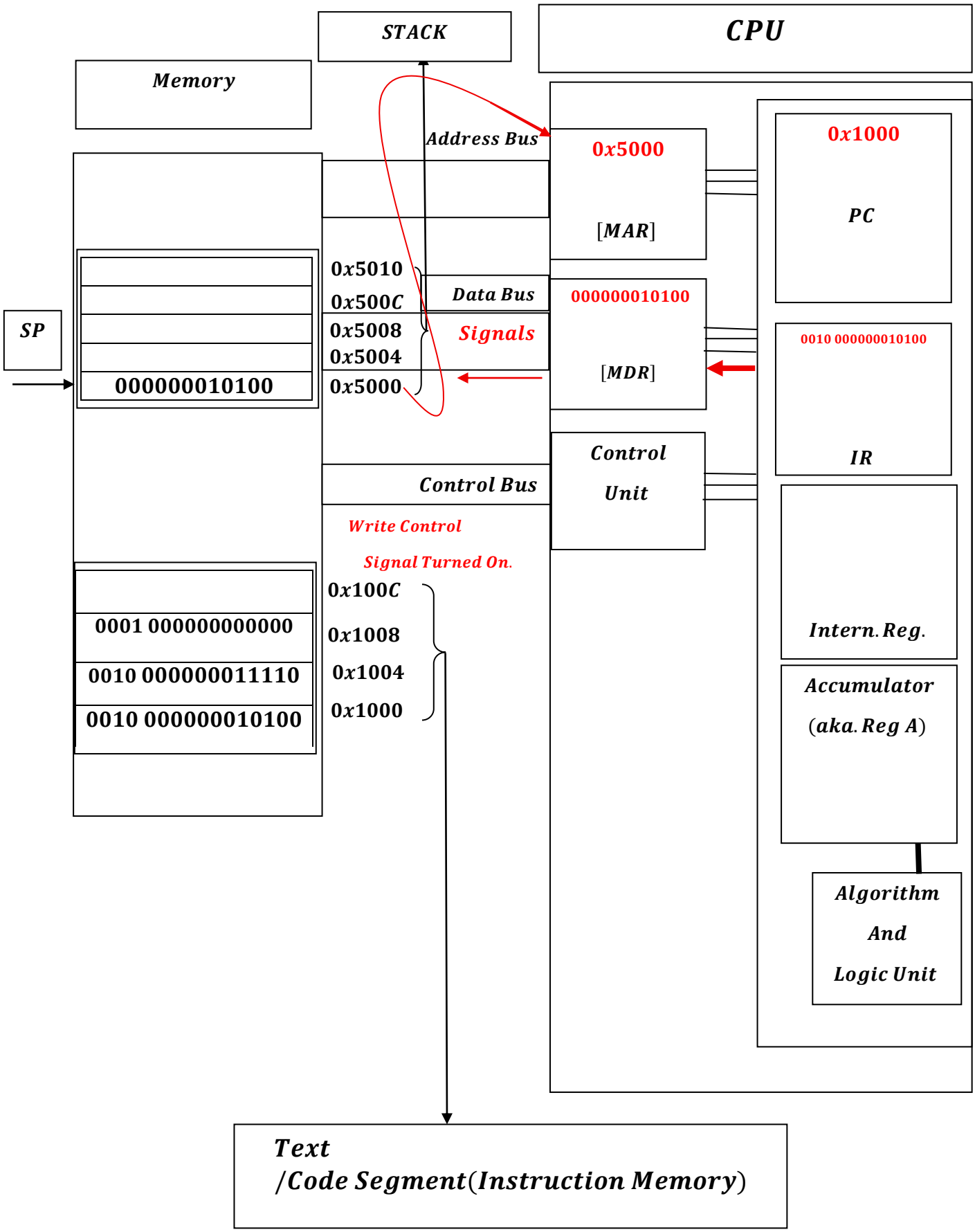
Control Unit (CU)

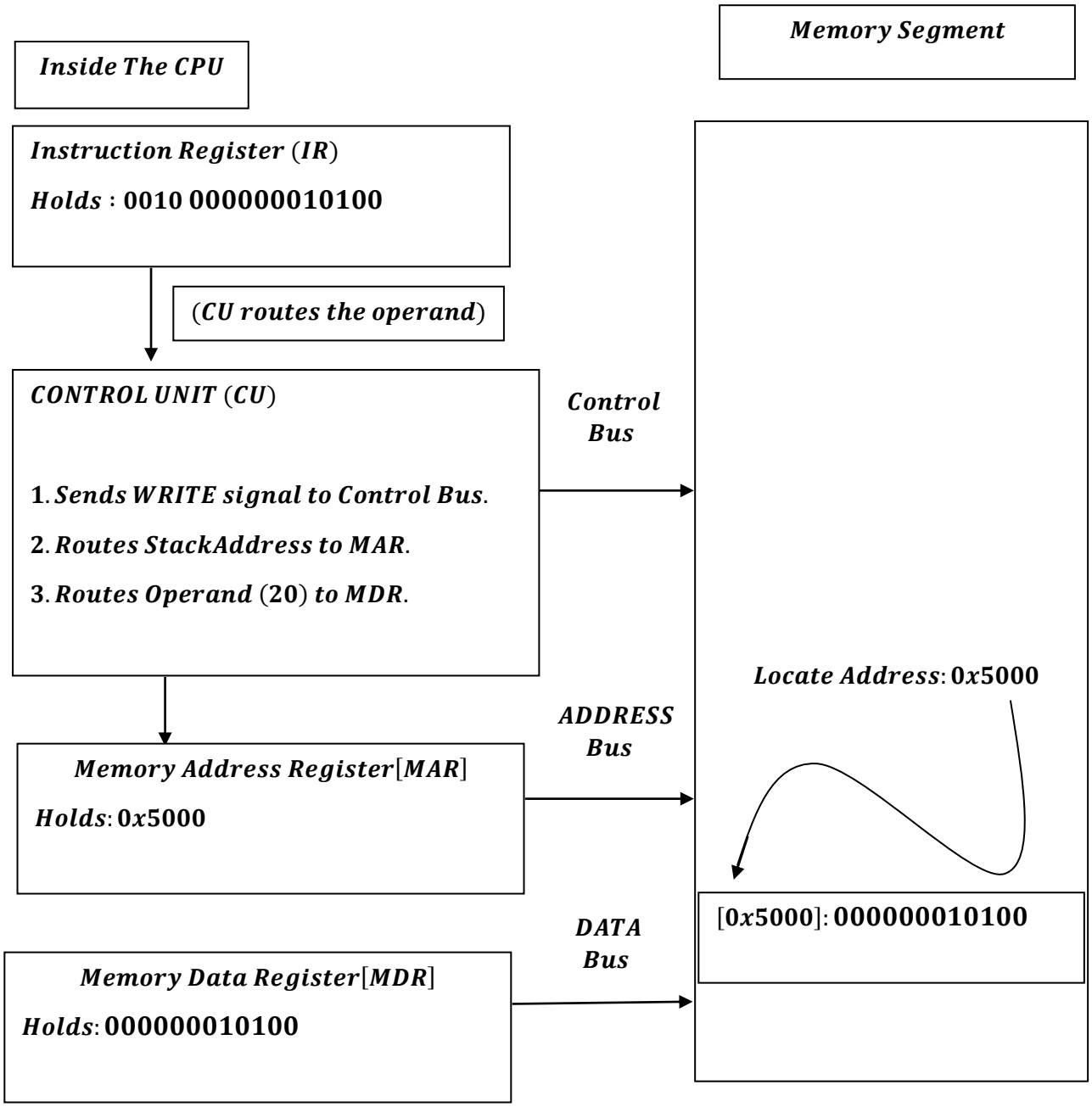
***Acts as translator*:**

1. *Extracts 4 – bit Opcode*: [0010] → "*Ah, this is PUSH*"

2. *Extracts 12 – bit Operand*: [000000010100] → "*Data is to Push is* 20"

Now that the instruction has safely reached the Instruction Register (IR) and the Control Unit (CU) has decoded it as a PUSH 20, the CPU shifts from the Fetch Phase to the Execution Phase.


STACK
CPU
Memory
Address Bus
0x5000
[MAR]
0x1000
PC
0x5010
0x500C
Data Bus
000000010100
SP
0x5008
Signals
0010 000000010100
0x5004
000000010100
0x5000
[MDR]
IR
Control Unit
Control Bus
Write Control
Signal Turned On.
0x100C
Intern. Reg.
0001 000000000000
0x1008
0010 000000011110
0x1004
Accumulator
(aka. Reg A)
0010 000000010100
0x1000
Algorithm
And
Logic Unit
Text
/Code Segment(Instruction Memory)

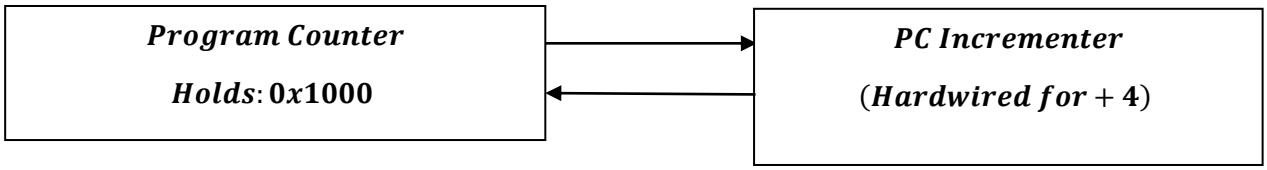


→ *CU looks at the Stack Pointer Register to see where the top of the stack currently is.*

→ *It takes that address and sends it directly to the Memory Address Register (MAR).*

→ *Once the data is successfully pushed out to the Data Bus, the hardware adder attached to the SP automatically calculates* $SP = SP + 4$.

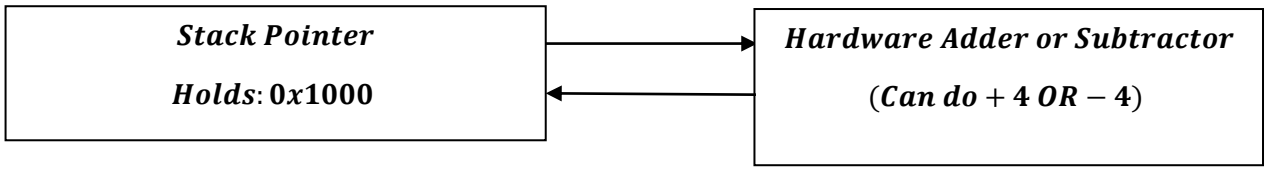
$Program\ COUNTER(Dedicated\ One - Way\ Incrementer)$



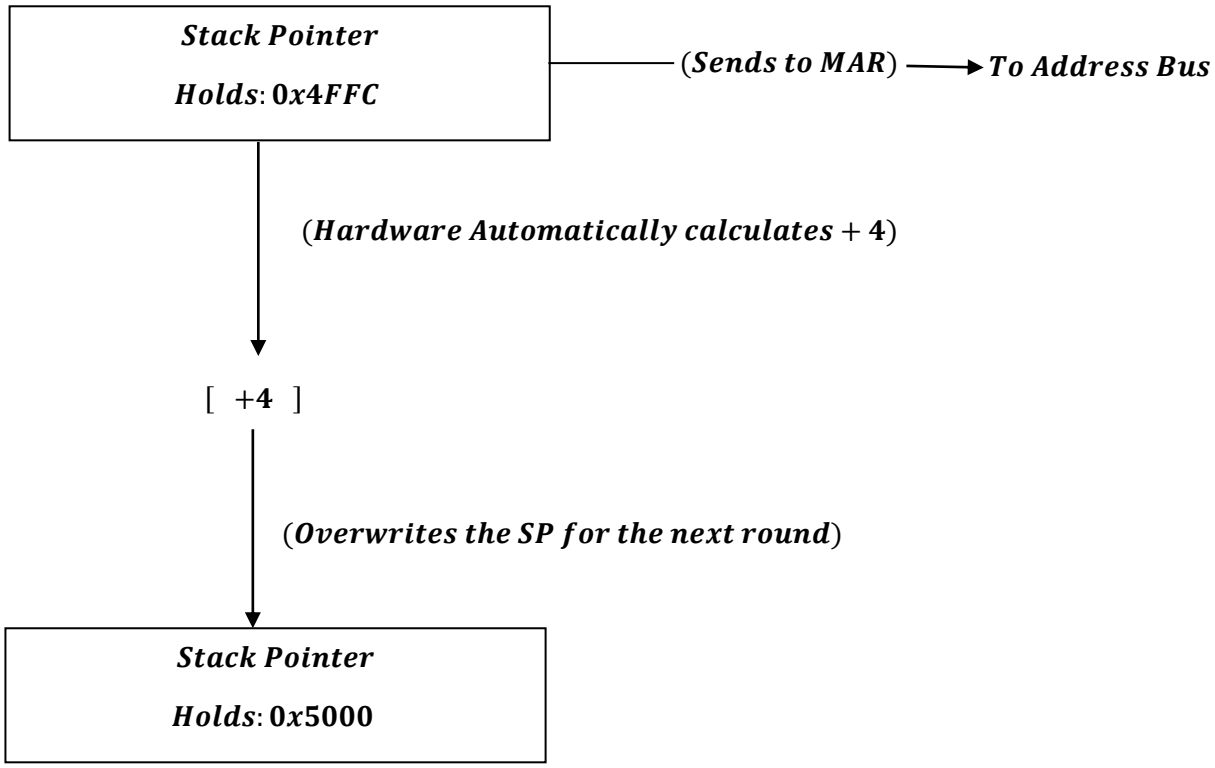
$(Always\ moves\ forward)$

==========

$Stack\ Pointer\ (Flexible\ Two - Way\ Adder\ or\ Subtractor)$

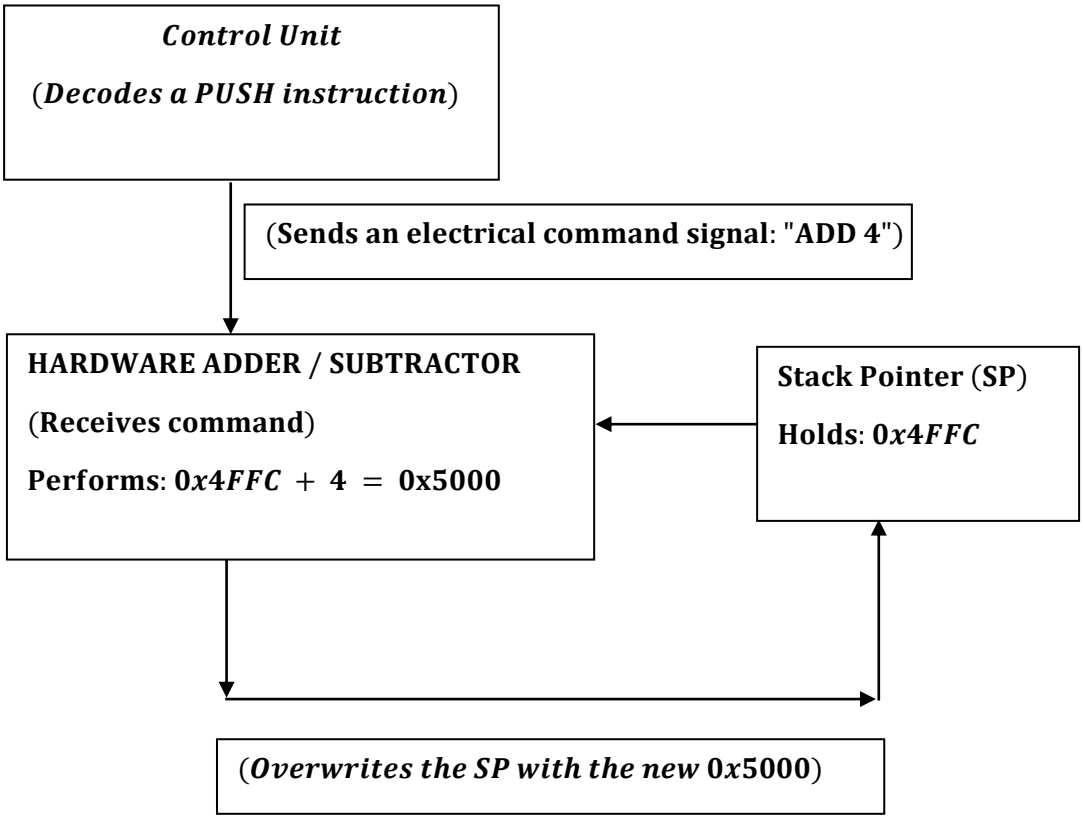


<u>During Push Operation in the Stack</u>



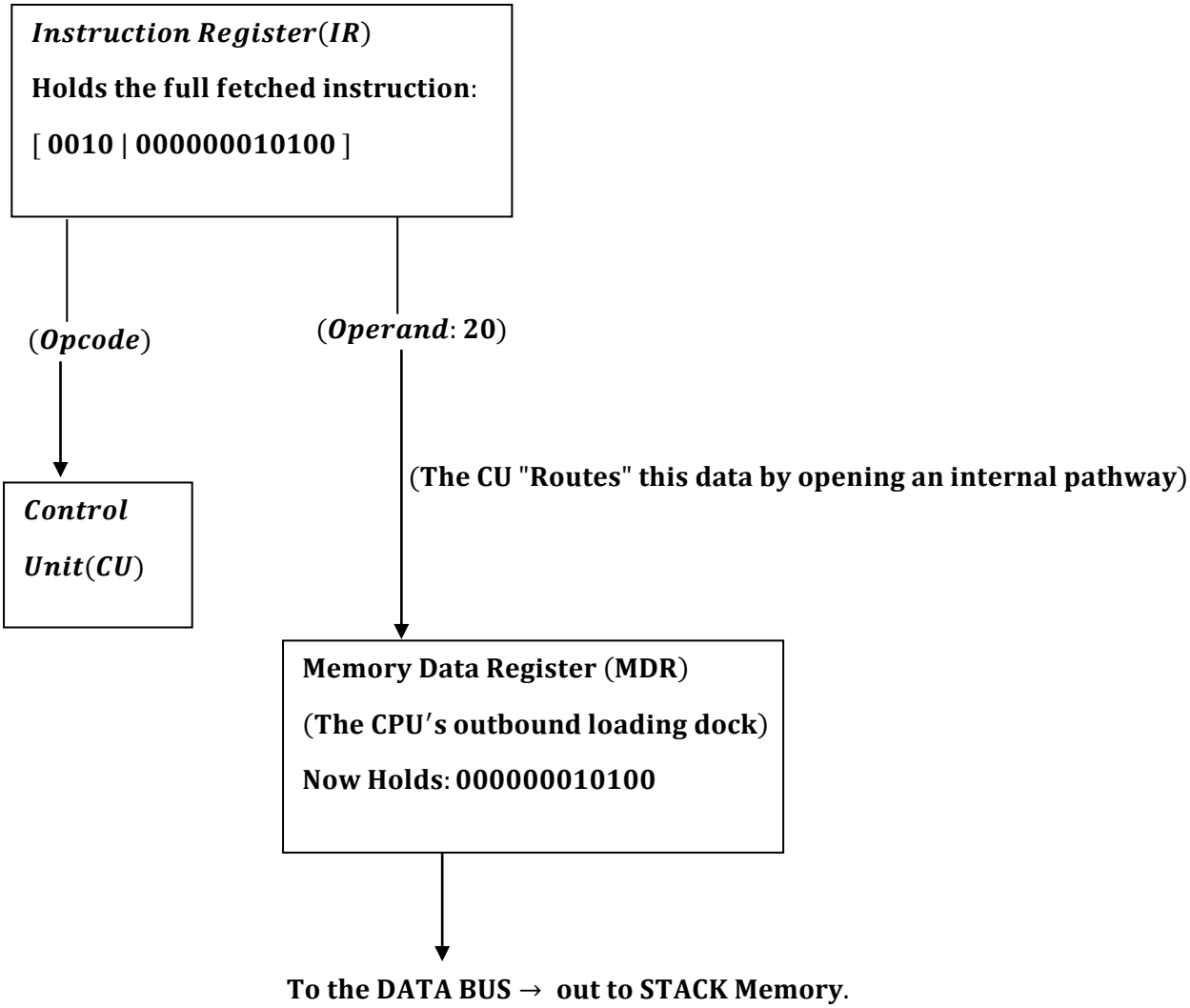
→ ***The Control Unit routes the Stack Address to the MAR directly (using the CPU's internal wiring), absolutely not through the Address Bus.***

→ ***The Control Unit acting as a Manager orders Stack Pointer to calculate the next Address.***



As soon as the MAR accepts this new data, whatever old address was sitting in the MAR is completely overwritten by the new address from the SP.

Routes Operand (20) to MDR.



The Control Unit looks at that 16 – bit string and splits it. It reads the first 4 bits (0010) and knows it means PUSH. It looks at the remaining 12 bits (000000010100) and knows this is the Operand – the actual raw data (the number 20) that needs to be pushed. The CU ignores the PUSH command part and grabs only that raw data.

Because the CPU needs to send this data out to the STACK memory, it cannot just throw it onto the Data Bus directly from the Instruction Register. All data leaving the CPU must go through the Memory Data Register (MDR).

``Routings`` *imply means the Control Unit sends electrical signals to open the internal logic gates, allowing those* 12 *bits* (000000010100)*to be copied from the Instruction Register directly down into the MDR.*

→ *SP(Stack Pointer) is* 0x5000 *, Data* 20 *is stored there.*

→ *Again REPEAT These process to PUSH* 30.

→ 1*st PC increments to* 0x1004 *overwriting the Previous Address.*

→ *MAR places the ADDRESS*: 0x1004 *from PC replacing(Overwriting) previous address stored.*

→ *CU activates READ Control Signal.*

→ *MAR then places the latched address* : 0x1004 *to ADDRESS Bus.*

→ *DATA placed on* 0x1004: 0010 000000011110, *is placed on DATA BUS and MDR recieves the data, overwriting the previous data.*

→ *Instruction Register(IR) recieves the data from the MDR overwriting the previous data.*

→ *Now instruction is PUSH* 30. *And CU activates Write Control Signal.*

→ *The Control Unit translates this and realizes the stack needs to grow to the next slot.*

→ *SP (Stack Pointer)register then add* 0x4 *to the previous stored memory address* 0x5000 + 0x4 = 0x5004.

→ *CU sends electrical signals to IR, where* 12 *bits*: 000000011110 *i.e.* 30 *and it is copied to MDR.*

→ *Then from MDR through DATA BUS , Data* 30 *is PUSHed to address*: 0x5004.

$If\ STACK\ is\ Upward\ growing\ then\ Adder\ is\ used\ for\ STACK\ Pointer\ Register\ and$

$If\ STACK\ is\ Downward\ growing\ then\ Subtracter\ is\ used\ for\ STACK\ Pointer\ Register.$

$Generally,$

$\rightarrow STACK\ POINTER\ Register\ contains:$

$\rightarrow One\ Binary\ Adder$

$\rightarrow Control\ Signal\ (ADD\ or\ SUB\ select\ line)$

$\rightarrow Constant\ Value(like\ 4\ byte)$

$Generally, Subtraction\ in\ Hardware\ happens:$

$A - B = A + (2's\ complement\ of\ B).$

$Hence\ subtracter\ may\ not\ be\ needed, but\ this\ also\ depend\ upon\ architecture\ we\ are\ using.$